Placement Portal Project Report

Student Details

Name: Vaidik

Roll Number: 24f2002098

Email: 24f2002098@ds.study.iitm.ac.in

About Me: I am a passionate developer with a keen interest in full-stack web development, particularly using Flask and SQLAlchemy. This project aims to streamline the placement process for educational institutes by providing a centralized platform for students, companies, and administrators.

Project Details

Project Title: Placement Management Portal

- **Problem Statement:**
  Educational institutions face challenges in managing placement drives, student applications, and company interactions. The manual process is often inefficient, lacks transparency, and leads to communication gaps. The goal is to build a web-based platform that automates the placement lifecycle, enabling students to apply to drives, companies to post opportunities, and administrators to oversee the entire process.
- **Approach:**
  The application is built using Flask as the backend framework with SQLAlchemy for ORM. It implements three distinct user roles: student, company, and admin.

Students can view and apply to approved drives, track their applications, and manage their profiles. Companies can create drives, review applicants, and update application statuses. Admins have oversight capabilities including user approval, blacklisting, and drive management. The system uses Flask-Login for authentication, Bootstrap for responsive UI, and Jinja2 for dynamic templating.

AI/LLM Declaration

I used ChatGPT (GPT-4) to assist in structuring the report, generating initial database schema ideas, and debugging some SQLAlchemy relationships. The extent of AI/LLM usage is around 10%, limited to code suggestions and documentation formatting. All final implementation logic, debugging, and integration were done manually.

Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Flask-Login	User authentication and session management
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
SQLite	Lightweight local database for storing data
Werkzeug	Security utilities (password hashing)
datetime	Date handling for deadlines and applications

Database Schema / ER Diagram

Tables:

- User – stores login credentials and role information

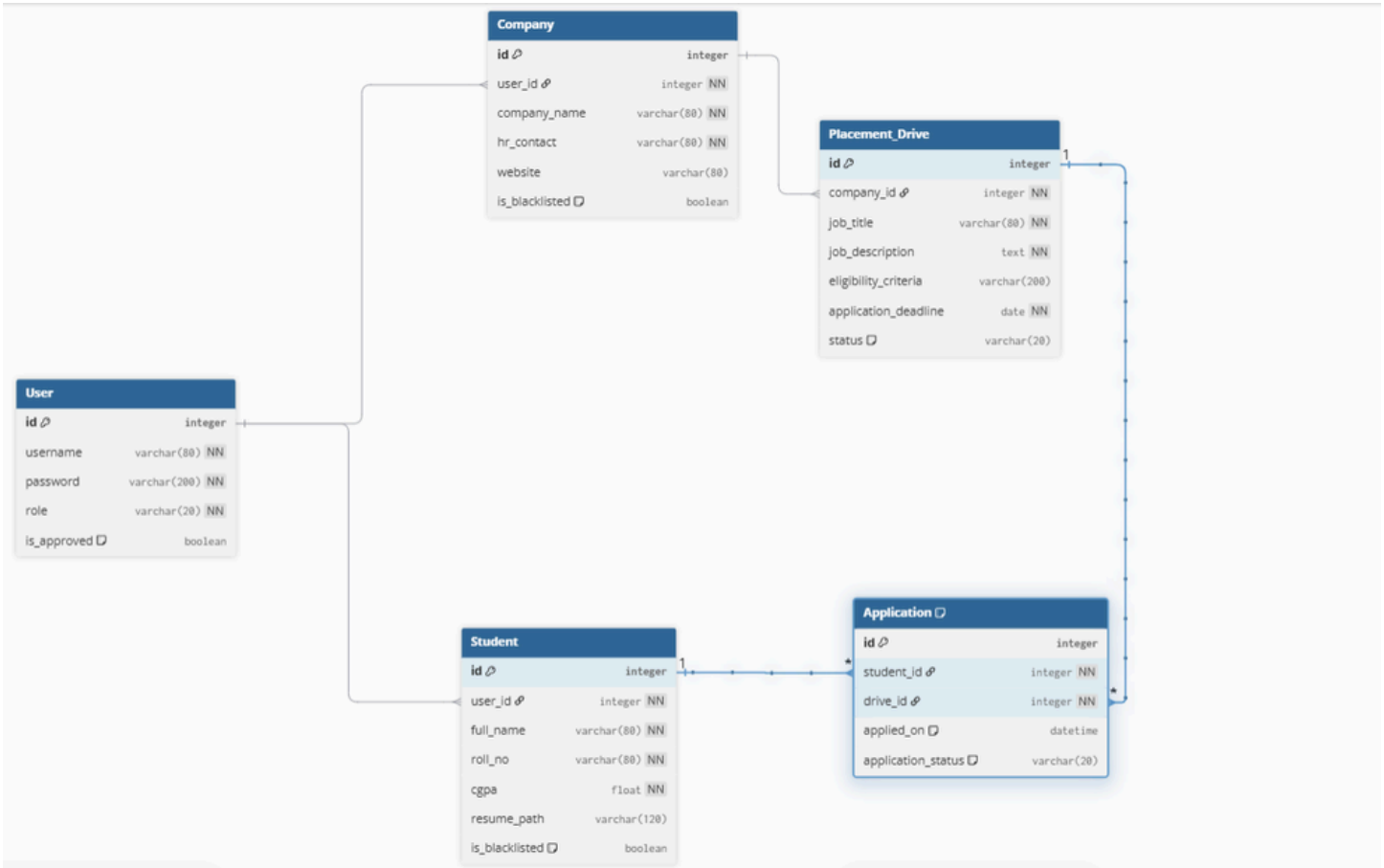
    - id (PK)
    - username (unique)
    - password (hashed)
    - role (admin, company, student)
    - is_approved (boolean)
- Student – extends User for student profiles
    - id (PK)
    - user_id (FK → User.id)
    - full_name
    - roll_no (unique)
    - cgpa
    - resume_path
    - is_blacklisted (boolean)
- Company – extends User for company profiles
    - id (PK)
    - user_id (FK → User.id)
    - company_name
    - hr_contact
    - website
    - is_blacklisted (boolean)
- Placement_Drive – represents a recruitment drive
    - id (PK)
    - company_id (FK → Company.id)
    - job_title
    - job_description
    - eligibility_criteria
    - application_deadline
    - status (Pending, Approved, Rejected)
- Application – tracks student applications to drives
    - id (PK)
    - student_id (FK → Student.id)

- drive_id (FK → Placement_Drive.id)
- applied_on (datetime)
- application_status (Applied, Shortlisted, Selected, Rejected)
- Unique constraint on (student_id, drive_id) to prevent duplicate applications

Relationships:

- One-to-One: User → Student, User → Company
- One-to-Many: Company → Placement_Drive
- One-to-Many: Placement_Drive → Application
- One-to-Many: Student → Application

ER Diagram (conceptual):



6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticate user and start session
/logout	GET	End user session
/register/student	POST	Register a new student
/register/company	POST	Register a new company (requires admin approval)
/student_dashboard	GET	View student dashboard with available drives and applications
/student/apply/<drive_id>	GET	Apply for a specific drive
/edit_profile	POST	Update student profile (CGPA, name)
/company/dashboard	GET	View company dashboard with posted drives
/company/create_	POST	Create a new

drive		placement drive
/company/view_applications/<drive_id>	GET	View applicants for a specific drive
/company/update_application/<app_id>/<status>	GET	Change application status (Shortlisted/Selected/Rejected)
/company/edit_drive/<drive_id>	POST	Edit existing drive details
/admin/dashboard	GET	Admin dashboard with statistics and management tabs
/blacklist/<role>/<id>	GET	Blacklist a student or company
/unblacklist/<role>/<id>	GET	Remove from blacklist
/admin/approve_company/<user_id>	GET	Approve a pending company registration
/admin/view_drive/<drive_id>	GET	View drive details (admin)
/admin/manage_drive/<drive_id>/<a	GET	Approve or reject a drive (admin)

ction>		
/admin/review_student/<id>	GET	View student details and application history
/review_company/<id>	GET	View company details and drives history

Architecture and Features

Architecture Overview:

- app.py – main Flask application with all route handlers.
- models.py – SQLAlchemy models defining database schema.
- templates/ – Jinja2 HTML files for rendering views.
- static/ – (optional) CSS, JS, and images.

The application follows the MVC pattern with Flask handling the controller layer, SQLAlchemy for models, and Jinja2 for views. Authentication is managed by Flask-Login, and sessions are used to maintain user state.

Implemented Features:

- Multi-role Authentication – Students, companies, and admin with distinct dashboards.
- User Registration & Approval – Companies require admin approval before posting drives.

- **Placement Drive Management – Companies can create, edit, and manage drives.**
- **Application Workflow – Students apply to approved drives; companies update application status (Applied → Shortlisted → Selected/Rejected).**
- **Admin Oversight – Admins can view pending approvals, blacklist users, review student/company profiles, and manage drives.**
- **Search Functionality – Admin can search students/companies by name or roll number.**
- **Blacklisting – Admins can blacklist students or companies, preventing them from logging in or participating.**
- **Deadline Enforcement – Applications are disabled after the deadline.**
- **Duplicate Prevention – Unique constraint on (student_id, drive_id) prevents multiple applications.**

Video Presentation

Drive Link:
Click Here For Video
(Accessible to all with "View" permission.)